

Bibbia di Algo

Matteo Barone

14 giugno 2022

Indice

1	Blocco 1	2
1.1	Complessità asintotica	2
1.2	Relazione di ricorrenza	3
2	Blocco 2	5
2.1	Array,liste	5
2.2	Pile,code,dizionari,algoritmi base su alberi	5
2.3	Algoritmi su alberi k-ari	5
3	Blocco 3	8
3.1	Analisi Algoritmo	8
3.2	Grafo	8

1 Blocco 1

1.1 Complessità asintotica

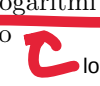
Principalmente calcolo di limiti utilizzando le seguenti formule:

$$f(x) \in O(g(x)) \rightarrow f(x) \leq g(x) \iff 0 \leq \lim_{x \rightarrow +\infty} f(x)/g(x) < \infty$$

$$f(x) \in \Theta(g(x)) \rightarrow c_1 g(x) \leq f(x) \leq c_2 g(x) \iff 0 < \lim_{x \rightarrow +\infty} f(x)/g(x) < \infty$$

$$f(x) \in \Omega(g(x)) \rightarrow g(x) \leq f(x) \iff 0 < \lim_{x \rightarrow +\infty} f(x)/g(x) \leq \infty$$

Nota bene:

- se ci troviamo in un caso $f(x)/g(x) \iff [\infty/\infty]$ allora utilizziamo de l'Hopital¹ n volte fino a cui non ci troviamo in un caso risolvibile $\rightarrow$ NON USARE DE L'HOPITAL SE ABBIAMO DEGLI ESPONENZIALI
- se abbiamo degli esponenziali, cercare di rendere le basi o gli esponenti uguali o eliminare gli esponenziali usando logaritmi $\rightarrow$ ricordarsi che i logaritmi non posso avere argomento negativo  $\log a (b) = c \rightarrow a^c = b$
- Ricordarsi che 0^+ e 0^- equivalgono a 0
- $\sin(x)$ e $\cos(x)$ hanno valore compreso tra -1 a 1 a ∞

¹Fare la derivata del numeratore e del denominatore separatamente

1.2 Relazione di ricorrenza

Esempio di relazione di ricorrenza iterativa:

$$T(n) = \begin{cases} c & n = 0 \\ T(n-1) + d & n \geq 1 \end{cases}$$

Il master theorem per partizioni costanti:

$$T(n) = \begin{cases} d & \text{se } n \leq m \leq h \\ \sum_{1 \leq i \leq h} a_i T(n-i) + cn^\beta & \text{se } n > m \end{cases}$$

Teorema. Se $c > 0, \beta \geq 0, a = \sum_{1 \leq i \leq h} a_i$ allora $\begin{cases} T(n) \in O(n^{\beta+1}) & \text{se } a = 1 \\ T(n) \in O(a^n n^\beta) & \text{se } a \geq 2 \end{cases}$

Il teorema si chiama teorema master per relazioni lineari a partizione costante.

Il master theorem per partizioni lineari:

Teorema. Se $a \geq 1; b \geq 2; c > 0; d, \beta \geq 0$, posto $\alpha = \log a / \log b$ allora :

$$\begin{cases} T(n) \in O(n^\alpha) & \text{se } \alpha > \beta \\ T(n) \in O(n^\alpha \log n) & \text{se } \alpha = \beta \\ T(n) \in O(n^\beta) & \text{se } \alpha < \beta \end{cases}$$

$$T(n) = \begin{cases} d & \text{se } n = 1 \\ aT(n/b) + cn^\beta & \text{se } n > 1 \end{cases}$$

Nota bene:

- Spesso c'è una sommatoria $\rightarrow$ ricordarsi di trasformarla usando la formula per la media ²
- Arriveremo ad avere un K che verrà trasformato in N in modo da trasformare $T(n)$ in $T(1)$ ³
- quando ci sono dei $(x \div y)^n$ usare la formula della progressione geometrica
- se ho $f(x) * O(g(x))$ allora $O(f(x) * g(x))$
- chiede di leggere un algoritmo, non è come gli esercizi del capitolo 2
- Leggi il file Relazioni di ricorrenza.pdf per una lista di tutti i casi $T(n)$ e i corrispondenti $\Theta(n)$

² $((\text{Elemento iniziale} + \text{elemento finale})/2) * \text{numero di elementi}$

³ovvero il caso base es. se $T(n \div 2^k)$ allora $k = \log_2 n$

2 Blocco 2

ATTENZIONE: Ricorda di testare i casi limite tipo struttura vuota o con solo un elemento

2.1 Array, liste

Cose da ricordare:

- Studiare il codice dei principali algoritmi di ordinamento
- Quasi sempre ricorsivo e si scorre fino al fondo

Segui questo template per scorrere la lista fino al fondo:

```
list Method(list l){
    if(l->next == NULL){ /* potrebbe esserci bisogno di scorrere fino a l
        == NULL */
        // ritorna cosa ti serve
    }else{
        list k = metodo(l->next);
        //fai cosa ti serve
        //se distruttivo sovrascrivi l->info con altro
        //se no lavora su k
    }
    return /*k o l per i motivi sopra oppure un valore se il metodo e' di
        un altro tipo*/
}
```

2.2 Pile, code, dizionari, algoritmi base su alberi

Leggere e capire le strutture presenti su moodle esami

2.3 Algoritmi su alberi k-ari

Visita in profondità (DFS): lungo i rami, dalla radice alle foglie.

Visita in ampiezza (BFS): per livelli, da quello della radice in poi.

Esempio visita in profondità (DFS) che NON UTILIZZA una pila:

```
TREE-DFS(k-Tree T)  
  visita T.key  
  C  $\leftarrow$  T.child  
  while C  $\neq$  nil do  
    TREE-DFS(C)  
    C  $\leftarrow$  S.sibling  
  end while
```

Il ciclo while controlla sui fratelli. Nel ciclo c'è sempre una chiamata ricorsiva per scendere di un livello. S è un typo, si intende C.
E' necessario avere una condizione if per la terminazione della chiamata ricorsiva

Esempio di visita in profondità(DFS), che UTILIZZA una pila:

```
S  $\leftarrow$  pila vuota  
Push(S, T)  
while S  $\neq$  la pila vuota do  
  T'  $\leftarrow$  Pop(S)  
  visita T'.key  
  for all C figlio di T' do  
    Push(S, C)  
  end for  
end while
```

Esempio di visita in ampiezza(BFS), che UTILIZZA una coda:

```
TREE-BFS( $k$ -Tree  $T$ )      ▷ pre:  $T$  non è vuoto
 $Q \leftarrow$  coda vuota
 $Enqueue(Q, T)$ 
while  $Q \neq$  la coda vuota do
     $T' \leftarrow Dequeue(Q)$ 
    visita  $T'.key$ 
    for all  $C$  figlio di  $T'$  do
         $Enqueue(Q, C)$ 
    end for
end while
```

La differenza tra la pila(anche nota come Stack) e la Coda(anche nota come Queue) è che la pila è LIFO⁴ mentre la coda è FIFO⁵

⁴Last In First Out ovvero ultimo che entra, primo che esce

⁵First In First Out ovvero primo che entra, primo che esce

3 Blocco 3

3.1 Analisi Algoritmo

- un for dentro a un for rende la complessità sempre $\theta(n^2)$ non importa se entrambi i cicli non partono da 0 e arrivano a n-1 (o da 1 a n)
- Per capire cosa fa un algoritmo è importante individuare e guardare l'IF principale e la sua condizione
- Per ridurre la complessità di due for annidati, spesso devo riordinare l'array
- Due chiamate ricorsive allo stesso livello rendono la relazione di ricorrenza $2T(n-k)$

3.2 Grafo

Saper fare Prim e Dijkstra.

- Ricordare che Prim non cerca la distanza minima tra due punti ma il costo minimo⁶ per collegare tutti i nodi.
- Dijkstra invece cerca il percorso minimo per arrivare dal punto A al punto B. Avrà quindi diversi alberi minimi e quasi nessuno di essi sarà ricoprente.⁷

Cose da ricordare sui Grafi:

- Un grafo bipartito è un grafo dove ogni arco ha i due nodi che lo compongono in due insiemi disgiunti tra loro⁸

⁶per costo minimo si intende la somma dei pesi degli archi minima

⁷per ricoprente si intende che il nuovo Grafo avrà ancora tutti i nodi del precedente e ogni nodo avrà almeno un nodo adiacente

⁸vedi esercizio 4 della simulazione su Algoritmi A 2021-2022 Teoria